

Sviluppo Web Essenziale in Python con Flask e Django

Introduzione allo Sviluppo Web in Python

Lo sviluppo web in Python consente di creare applicazioni web utilizzando il linguaggio Python come base sia per la logica backend che per generare contenuti dinamici. In questa lezione tratteremo due popolari framework Python per il web: **Flask** e **Django**. Flask è un *microframework* leggero e flessibile, mentre Django è un *full-stack framework* più strutturato e "batteries-included". Entrambi permettono di gestire routing (URL), template HTML, form, interazione col database e molto altro, ma con filosofie diverse.

Pur assumendo solide basi di Python, introdurremo gradualmente concetti chiave dello sviluppo web. Vedremo come definire rotte URL e gestire richieste HTTP (GET, POST), come utilizzare i template per generare interfacce HTML dinamiche, come validare input dell'utente tramite form, interagire con un database usando un ORM (Object-Relational Mapping), costruire API REST e gestire l'autenticazione di base. Infine, esploreremo anche come integrare il frontend tramite AJAX/JSON e distribuire (*deploy*) le applicazioni su servizi online gratuiti. Iniziamo con una panoramica di Flask e Django e come creare il classico "Hello, World" in ciascuno.

```
# Esempio minimale di applicazione web in Flask (Hello World)
from flask import Flask
app = Flask(__name__)                                # Crea una nuova applicazione Flask
@app.route('/')                                       # Definisce una rotta all'URL radice
"/"
def indice():
    return "Ciao, Mondo da Flask!"                  # Risposta testuale per la richiesta
HTTP GET "/"
if __name__ == '__main__':
    app.run(debug=True)                             # Avvia il server di sviluppo Flask
(in modalità debug)
```

Flask: Un Microframework Leggero

Flask è un framework minimalista che fornisce gli elementi essenziali per creare un'applicazione web, lasciando al programmatore la libertà di scegliere componenti aggiuntivi secondo necessità. Essendo leggero, Flask non impone una struttura rigida: un'app semplice può stare in un unico file Python. Si basa su Werkzeug (per il routing e server WSGI) e Jinja2 (per i template HTML). Questa flessibilità rende Flask ideale per progetti piccoli o medi, API leggere o prototipi rapidi, in cui si vuole controllo totale sugli elementi da includere.

In Flask si aggiungono funzionalità man mano: ad esempio, se serve un database si può integrare un'ORM come SQLAlchemy, per i form si può usare Flask-WTF, per l'autenticazione Flask-Login, ecc. La comunità di Flask offre molte estensioni. Per iniziare, definiremo alcune rotte con le relative funzioni di

vista. Ogni rotta in Flask si registra con un decoratore `@app.route`. Possiamo anche configurare alcuni aspetti globali dell'applicazione tramite `app.config`. Nell'esempio seguente, abilitiamo la modalità debug e aggiungiamo una rotta con un parametro dinamico per mostrare quanto sia facile estendere l'app.

```
app.config['DEBUG'] = True                # Abilita la modalità
debug (riavvio automatico, errori dettagliati)
@app.route('/ciao/<nome>')                # Definisce una rotta
    dinamica con parametro "nome"
def saluto(nome):
    return f"Ciao, {nome} da Flask!"      # Usa il parametro
    dinamico nella risposta
# Aggiungendo rotte è semplice ampliare la funzionalità dell'app Flask
```

Django: Un Framework Full-Stack

Django è un framework **full-stack** che fornisce un'architettura completa e "tutto compreso" per applicazioni web di qualsiasi dimensione. Django segue il principio "batteries included", offrendo out-of-the-box un ORM potente, sistema di template, gestione dei form, autenticazione utente, pannello di amministrazione, e molto altro. Questo comporta una struttura di progetto più articolata rispetto a Flask: Django impone di separare le componenti in file e moduli (settings, URL, viste, modelli, template, ecc.), favorendo le buone pratiche e la scalabilità sin dall'inizio.

Con Django normalmente si crea un progetto e all'interno di esso una o più app. Ogni app Django è un modulo con modelli, viste, template propri che poi vengono integrati nel progetto. Django è ottimo per applicazioni complesse e progetti a lungo termine, dove la coerenza e le funzionalità integrate (come il pannello admin o la gestione utenti) accelerano lo sviluppo. Di seguito vediamo un esempio minimale di come Django mappa un URL a una vista. A differenza di Flask, la configurazione delle rotte avviene in un file separato (`urls.py`) e le funzioni di vista sono definite in `views.py`. Nell'esempio, definiamo una vista semplice che restituisce un saluto e la colleghiamo a un URL nel `urlpatterns`.

```
# file: myapp/views.py (Django)
from django.http import HttpResponse
def saluto(request):
    return HttpResponse("Ciao, Mondo da Django!") # Risposta HTTP semplice
    con un saluto

# file: myapp/urls.py (Django)
from django.urls import path
from . import views
urlpatterns = [
    path('', views.saluto, name='saluto')          # Associa l'URL radice ''
    alla vista saluto
]
# Questa configurazione va inclusa nell'urls.py principale del progetto
```

Struttura del Progetto: Flask vs Django

Un progetto Flask tipicamente ha una struttura semplice. Per applicazioni piccole, un singolo file `app.py` può contenere l'applicazione, le rotte e la logica. È possibile organizzare un'app Flask in più moduli, ma non c'è una rigida imposizione da parte del framework. Al contrario, un progetto Django ha una struttura predefinita generata dallo strumento `django-admin`. Django separa nettamente il progetto (configurazione globale) dalle applicazioni (funzionalità specifiche). Questo incoraggia modularità e riusabilità: si possono avere più app Django in un progetto, ciascuna focalizzata su un dominio specifico (es. blog, forum, ecc.).

Di seguito confrontiamo in modo semplificato le due strutture. Notiamo che Django crea automaticamente file come `settings.py` (configurazioni), `urls.py` (routing), `models.py` (definizioni del database ORM), `views.py` (logica delle viste), ecc. Flask invece non genera nulla automaticamente: la struttura è a discrezione dello sviluppatore, che può iniziare con uno script Python ed eventualmente organizzare cartelle per template e static file. Una convenzione comune in Flask è avere una cartella `templates/` per i file HTML e `static/` per file statici (CSS, JS, immagini).

```
# Struttura minima di un progetto Flask:
my_flask_app/
├── app.py           # Applicazione Flask principale (app, rotte, ecc.)
├── templates/       # (Facoltativo) Template HTML
└── static/          # (Facoltativo) File statici (CSS, JS, immagini)

# Struttura di un progetto Django:
my_django_project/
├── manage.py        # Utility command-line per task Django
├── my_django_project/ # Directory del progetto
│   ├── settings.py   # Configurazione del progetto (database, app
│   │               installate, ecc.)
│   ├── urls.py       # URL globali del progetto (include gli URL delle
│   │               app)
│   ├── asgi.py, wsgi.py # Entry-point ASGI/WSGI per il deploy
│   └── __init__.py
├── myapp/            # Una applicazione Django all'interno del progetto
│   ├── models.py     # Definizione dei modelli di database per questa app
│   ├── views.py      # Definizione delle viste (logica di risposta alle
│   │               richieste)
│   └── urls.py       # URL specifici dell'app (da includere nel urls.py
│   │               del progetto)
│   ├── forms.py      # (Opzionale) Definizione di form
│   ├── templates/    # Template HTML specifici dell'app
│   └── ... (altri file come admin.py, apps.py, tests.py)
```

Flask: Definizione di Rotte

Il **routing** si riferisce al collegamento tra URL specifici e funzioni che generano le risposte. In Flask, il routing è gestito tramite decorator: si utilizza `@app.route()` sopra una funzione per associarla a uno specifico percorso URL. Quando arriva una richiesta HTTP per quell'URL, Flask esegue la funzione

associata (chiamata *vista*). Nel decoratore si può specificare il percorso (ad esempio `'/'` per la homepage) e anche i metodi HTTP consentiti (GET, POST, ecc., di default solo GET).

Una volta definita la rotta, all'interno della funzione di vista possiamo utilizzare la logica Python per processare eventuali parametri, interagire con il database o altre operazioni, e infine restituire un valore (stringa, oggetto Response o risultato di `render_template`). Flask si occuperà di trasformare il valore restituito in una risposta HTTP valida. Nell'esempio seguente, definiamo due rotte Flask semplici: una per la homepage e una per una pagina "info". Mostriamo come ogni funzione associata a una rotta ritorna del contenuto da inviare al client.

```
from flask import Flask
app = Flask(__name__)

@app.route('/')                                # Rotta per la homepage (URL
radice '/')
def homepage():
    return "<h1>Homepage</h1>Benvenuto!"    # Restituisce HTML semplice come
risposta

@app.route('/info')                            # Rotta per "/info"
def info():
    return "Pagina di informazione generica." # Restituisce testo semplice
per /info
```

Flask: Rotte con Parametri URL

Spesso le rotte hanno segmenti variabili per identificare risorse o passare informazioni (es. `/utente/123` per la pagina dell'utente con ID 123). Flask supporta parametri dinamici nelle URL: basta indicare nel percorso la porzione variabile racchiusa in `< >`. Dentro le `< >` possiamo specificare anche un convertitore di tipo (stringa di default, ma possiamo usare `<int:...>` per interi, `<float:...>` etc.). La funzione di vista corrispondente riceverà i valori dei parametri come argomenti.

Nell'esempio definiremo una rotta dinamica per profili utente con un ID numerico, e una per un saluto con un nome (stringa). All'interno della funzione, possiamo usare questi parametri per personalizzare la risposta. Flask effettua anche il casting del parametro al tipo specificato (ad esempio `int`), e se il tipo non corrisponde (es. si passa "abc" dove è atteso un int), restituisce automaticamente un errore 404. Nei casi in cui serva, si possono definire convertitori personalizzati, ma qui usiamo quelli base.

```
@app.route('/utente/<int:user_id>')          # Rotta dinamica: user_id sarà
convertito in int
def profilo_utente(user_id):
    # Ad esempio potremmo cercare l'utente nel database usando l'ID
    return f"Profilo dell'utente #{user_id}"

@app.route('/saluto/<nome>/')                  # Rotta dinamica: nome come
stringa (predefinito)
def saluto(nome):
    return f"Ciao {nome}, benvenuto nel sito!"
```

```
# Esempi URL: "/utente/5" chiamerà profilo_utente(5); "/saluto/Marco/"
chiamerà saluto("Marco")
```

Django: URLconf e Routing di Base

In Django, il routing è gestito tramite un sistema di URL configuration (URLconf). Invece di decoratori, Django utilizza delle liste `urlpatterns` in cui ogni elemento associa un pattern di URL a una vista. Questo avviene tipicamente nel file `urls.py`. Le voci di `urlpatterns` possono utilizzare il metodo `path()` (o `re_path` per regex avanzate) per definire i percorsi. A differenza di Flask, in Django la funzione di vista riceve sempre almeno un parametro `request` (l'oggetto `HttpRequest`) e i parametri URL definiti nel pattern.

Ogni progetto Django ha un `urls.py` principale (creato con il progetto) che spesso include gli URL delle app. Ogni app può avere il suo `urls.py` modulare. Questo approccio modulare rende facile combinare diverse app con i propri URL. Nell'esempio seguiamo un semplice scenario: configuriamo un URL fisso e uno con parametro. In `urlpatterns` mappiamo `'info/'` a una vista `info` e `'saluto/<str:nome>/'` a una vista `saluto`. Notare l'uso di `<str:nome>` per indicare un parametro stringa chiamato nome (analogamente, `<int:id>` per interi, ecc.). Le viste corrispondenti verranno implementate in `views.py` della stessa app.

```
# file: myapp/urls.py (Django - routing dell'applicazione)
from django.urls import path
from . import views
urlpatterns = [
    path('info/', views.info, name='info'),          # URL fisso ->
    vista info()
    path('saluto/<str:nome>/', views.saluto, name='saluto') # URL con
    parametro 'nome' (stringa) -> vista saluto()
]

# file: myapp/views.py (Django - viste corrispondenti)
from django.http import HttpResponse
def info(request):
    return HttpResponse("Pagina di informazioni generiche.")
def saluto(request, nome):
    return HttpResponse(f"Ciao {nome}, benvenuto sul sito Django!")
]
# Queste voci vanno incluse nel urls.py del progetto: es. path('',
include('myapp.urls'))
```

Django: Parametri nelle URL

Django permette di catturare parametri dalle URL definendo i segmenti dinamici nei pattern di routing. Nel metodo `path()`, la sintassi `<tipo:nome>` consente di specificare il nome del parametro che verrà passato alla vista e, opzionalmente, il tipo (es. `str`, `int`, `slug` ecc.). Come visto, `<str:nome>` passa una stringa, `<int:id>` passa un intero (o dà 404 se non è un numero). I parametri catturati vengono forniti come argomenti alla funzione di vista corrispondente.

Riprendendo l'esempio, la nostra vista `saluto(request, nome)` riceve il parametro `nome` estratto dall'URL. Django assicura la corrispondenza fra nome nel pattern e nome del parametro nella funzione. È importante notare che i pattern Django non includono la parte di dominio o prefisso del sito; spesso configuriamo i pattern a partire da una base definita altrove (ad esempio in `include()`). Inoltre, Django consente di assegnare un nome univoco ad ogni pattern di URL (con il parametro `name`), per potervi fare riferimento altrove (nei template o nel codice) tramite funzioni di inversione (`reverse`). Vediamo un altro esempio di parametro numerico per chiarezza:

```
# Aggiungiamo un URL con parametro numerico (ID articolo) in urls.py
urlpatterns += [
    path('articolo/<int:id>/', views.articolo_dettaglio,
         name='articolo_dettaglio')
]

# In views.py definiamo la vista corrispondente
def articolo_dettaglio(request, id):
    # Esempio: recupero di un articolo dal DB (pseudocodice)
    # articolo = Articolo.objects.get(pk=id)
    return HttpResponse(f"Dettagli dell'articolo {id}")
# Se l'URL richiesto è "/articolo/5/", Django chiamerà
articolo_dettaglio(request, id=5)
```

Flask: Generazione di URL Dinamici

Hardcodare URL nel codice o nei template è considerato cattiva pratica perché rende meno manutenibile l'app (ogni modifica ai percorsi richiederebbe di aggiornare manualmente tutte le occorrenze). Flask fornisce la funzione `url_for()` per generare URL in base al nome della funzione di vista. Ciò consente di costruire link dinamicamente e in modo sicuro, usando i nomi delle funzioni e i parametri, anziché stringhe fisse di URL. `url_for` tiene conto anche del *prefix* dell'app o dello script name, se l'app non è in root.

In Flask, la funzione di vista ha un nome (di default, il nome Python della funzione). `url_for('nome_funzione', **parametri)` restituirà la stringa di URL corrispondente. Ad esempio, se abbiamo definito `@app.route('/utente/<int:user_id>', methods=['GET'])` associata a `def profilo_utente(...)`, potremo ottenere l'URL per un certo utente con `url_for('profilo_utente', user_id=42)` - ottenendo `"/utente/42"`. Questo è utile sia nelle viste (per fare redirect) che nei template (per creare link HTML). Ecco un breve esempio di utilizzo di `url_for` all'interno di un contesto di richiesta Flask:

```
from flask import url_for, redirect

@app.route('/vai-al-profilo')
def vai_al_profilo():
    # Reindirizza l'utente loggato al proprio profilo
    user_id = 7 # ipotizziamo di avere l'ID 7
    profilo_url = url_for('profilo_utente', user_id=user_id) # Genera URL
    "/utente/7"
    return redirect(profilo_url) # Effettua
```

```
redirect a quella URL
```

```
# Nel template Jinja2 si usa url_for per link dinamici, ad esempio:  
# <a href="{% url_for('profilo_utente', user_id=7) %}">Vai al tuo profilo</a>
```

Django: Reverse URL e Tag URL

Analogamente, Django scoraggia l'uso di URL hardcoded. Al loro posto, offre la funzione `reverse()` (nel codice Python) e il tag template `{% url %}` nei template, per ottenere l'URL associato a un certo nome di percorso. Quando definisce un pattern in `urls.py`, tipicamente si assegna un `name` (come abbiamo fatto con `'saluto'`, `'info'` ecc.). Questo nome consente di riferirsi all'URL senza conoscerne la stringa esatta. Per esempio, in un template HTML Django possiamo creare un link al saluto con `href="{% url 'saluto' 'Mario' %}"`, passando anche eventuali parametri.

Nel codice Python (ad esempio in una vista o altrove), si può usare `from django.urls import reverse` per ottenere la stringa URL. `reverse('nome_pattern', args=[parametri])` oppure `reverse('nome_pattern', kwargs={'nome': 'valore'})` restituirà l'URL corrispondente. Questo è molto utile per fare redirect o per costruire email con link, ecc. Ecco un esempio di utilizzo:

```
from django.urls import reverse  
# Supponiamo di avere un pattern con name='saluto' che richiede un argomento  
# 'nome'  
url = reverse('saluto', args=['Mario'])      # Restituisce "/saluto/Mario/"  
print(url)                                  # Stamperà qualcosa come "/  
saluto/Mario/"  
  
# In un template Django, per creare un link a 'saluto' passando l'argomento:  
# <a href="{% url 'saluto' 'Mario' %}">Clicca qui per il saluto a Mario</a>  
# Se il pattern include app_name, si userebbe 'appname:saluto' come nome  
# completo nel tag.
```

Flask: Template con Jinja2

Generare HTML dinamicamente concatenando stringhe manualmente sarebbe scomodo e soggetto a errori. Flask utilizza **Jinja2** come motore di template per separare la logica dalla presentazione. I file template sono tipicamente file HTML con estensione `.html` che contengono segnaposto e direttive speciali. Ad esempio, in Jinja2, `{{ variabile }}` inserisce il valore di una variabile, e le strutture di controllo come `{% if ... %}` o `{% for ... %}` permettono di inserire logica di presentazione (condizioni, cicli) direttamente nel template.

Per usare i template in Flask, creiamo una cartella `templates/` e vi salviamo i file HTML. La funzione Flask `render_template('nome_template.html', **context)` carica il template e lo rende popolando i segnaposto con il contesto fornito (variabili passate come parametri). Nell'esempio seguente, definiamo un semplice template HTML con Jinja2 che saluta l'utente per nome, e una vista Flask che lo utilizza. Notare che `render_template` cerca i file nella cartella `templates` per default.

```
# file: templates/saluto.html (Template Jinja2 per Flask)
<!DOCTYPE html>
<html>
  <body>
    <h1>Ciao, {{ nome }}</h1>      <!-- Inserisce il valore della variabile
'nome' -->
    <p>Benvenuto nella pagina Flask di esempio.</p>
  </body>
</html>

# file: app.py (estratto - utilizzo del template)
from flask import render_template
@app.route('/pagina/<nome>')
def pagina(nome):
    return render_template('saluto.html', nome=nome) # Passa 'nome' al
template per il rendering
```

Django: Template e Rendering

Django ha un proprio motore di template, simile a Jinja2 (in effetti la sintassi con {{ }} e {% %} è molto simile). I template Django supportano variabili, filtri, tag di controllo (if, for), e prevengono vulnerabilità comuni con l'escaping automatico dell'output HTML. Per utilizzare un template in Django, si usa la funzione `render(request, 'template.html', context)` all'interno di una vista. I template vengono cercati nelle directory specificate nelle impostazioni (per default, ciascuna app può avere una cartella `templates/` inclusa automaticamente).

Nell'esempio sottostante, creiamo un template per un saluto analogo a quello visto in Flask, e lo usiamo in una vista Django. Osserviamo che Django richiede di includere manualmente la protezione CSRF nei form (via tag `{% csrf_token %}`, che vedremo meglio in sezione form). In questa semplice pagina di saluto, non c'è form quindi non è necessario. La vista passa un dizionario con `'nome': nome` come contesto al template, e Django si occupa di renderizzare il file HTML con quel contesto.

```
<!-- file: templates/saluto.html (Template Django) -->
<!DOCTYPE html>
<html>
  <body>
    <h1>Ciao, {{ nome }}</h1>
    <p>Benvenuto nella pagina Django di esempio.</p>
  </body>
</html>

# file: views.py (Django - vista che utilizza il template)
from django.shortcuts import render
def pagina(request, nome):
    return render(request, 'saluto.html', {'nome': nome}) # Passa il
contesto con 'nome' al template
```


Template Inheritance e Layout

Sia Jinja2 (Flask) che il motore template Django supportano l'**ereditarietà dei template**, una potente funzionalità per evitare ripetizioni e mantenere un layout consistente. Si realizza definendo un template base (es. `base.html`) con sezioni "segnaposto" (blocchi) che i template figli possono riempire. Ad esempio, un `base.html` potrebbe definire l'intelaiatura HTML comune (header, footer) e un blocco content centrale. Ogni pagina specifica estenderà `base.html` ed inserirà il proprio contenuto in quel blocco.

Questo approccio consente di aggiornare in un unico punto elementi comuni (menu, stile, script) e fa sì che le pagine derivate ereditino automaticamente le modifiche. Vediamo un esempio generico. Creiamo un layout base con un blocco content, e poi una pagina che estende il base. Questa sintassi vale per Django (come mostrato) e analogamente in Flask con Jinja2, poiché i costrutti `{% extends %}` e `{% block %}` sono praticamente gli stessi.

```
<!-- file: templates/base.html -->
<html>
  <head><title>{% block titolo %}Mio Sito{% endblock %}</title></head>
  <body>
    <nav>...barra di navigazione...</nav>
    <section>
      {% block content %}{% endblock %}    <!-- Contenuto variabile nelle
pagine figlie -->
    </section>
    <footer>Copyright 2025 - Il Mio Sito</footer>
  </body>
</html>

<!-- file: templates/pagina.html -->
{% extends "base.html" %}
{% block titolo %}Pagina Esempio - Mio Sito{% endblock %}
{% block content %}
  <h1>Benvenuto!</h1>
  <p>Questa è una pagina di esempio che utilizza il layout base.</p>
{% endblock %}
```

Template: Condizioni e Cicli

Nei template possiamo inserire un minimo di logica di presentazione tramite **condizionali** e **cicli**. Ciò consente, ad esempio, di mostrare un messaggio solo se l'utente è autenticato, oppure di iterare su una lista di elementi per creare una tabella o un elenco HTML. È importante mantenere la logica nei template limitata alla presentazione (evitando di eseguire calcoli complessi o accessi al database dal template – queste operazioni vanno fatte nella vista e passate come dati). Sia Jinja2 che il motore Django usano sintassi simili: `{% if %} ... {% endif %}` per le condizioni e `{% for item in lista %} ... {% endfor %}` per i cicli.

Nell'esempio qui sotto, supponiamo di avere una lista di utenti da visualizzare. Il template itera su ogni utente e lo mostra in un elenco HTML. All'interno del ciclo, usiamo un `if` per evidenziare magari l'utente attivo. Inoltre, evidenziamo il concetto che i template proteggono automaticamente

dall'iniezione di codice indesiderato: se una variabile contiene `<script>...`, verrà escape-ata (es. `<` diventa `<`) per prevenire XSS, a meno che non si marchi esplicitamente come sicuro.

```
<ul>
  {% for utente in utenti %}
    <li>
      {% if utente.is_active %}
        <strong>{{ utente.nome }}</strong>  <!-- Utente attivo in grassetto
-->
      {% else %}
        {{ utente.nome }}
      {% endif %}
    </li>
  {% endfor %}
</ul>

<!-- Questo genererà una <li> per ogni utente nella lista 'utenti',
evidenziando quelli attivi -->
```

Flask: Gestione di Richieste GET/POST

Un server web deve gestire diversi metodi HTTP. **GET** è usato tipicamente per richieste di lettura (ottenere una pagina, un dato), **POST** per inviare dati (form submission), e ci sono anche PUT, DELETE, ecc. In Flask, per permettere a una funzione di vista di rispondere a metodi diversi, si specifica il parametro `methods` nel decoratore `@app.route`. Se non si indica nulla, la vista risponde solo a GET. All'interno della funzione, possiamo controllare `request.method` per differenziare il comportamento tra GET e POST.

Il modulo `flask` fornisce l'oggetto globale `request` (da importare) che contiene dati della richiesta corrente: `request.args` per parametri in query string (GET), `request.form` per dati inviati via form (POST), `request.files` per file upload, ecc. Nell'esempio seguente, implementiamo una singola rotta per gestire sia la visualizzazione di un form (GET) che la sua elaborazione (POST). Con una richiesta GET mostriamo un semplice form HTML; con POST leggiamo i dati inviati e restituiamo una conferma. Nota: Stiamo restituendo direttamente stringhe HTML per brevità, ma in pratica useremmo `render_template` per servire pagine più complesse.

```
from flask import request, render_template

@app.route('/invia', methods=['GET', 'POST'])
def invia_dati():
    if request.method == 'POST':
        nome = request.form['nome']          # Ottiene il campo 'nome'
        inviato tramite form
        return f"Dati ricevuti: {nome}"      # Risposta dopo l'invio
        (conferma)
    else:
        # Per GET, mostriamo un semplice form HTML (in realtà si potrebbe
        usare un template separato)
        return ""
```

```

        <form method="POST">
            Nome: <input type="text" name="nome">
            <input type="submit" value="Invia">
        </form>
    """
# Buona pratica: per form reali usare render_template per separare HTML e
logica

```

Django: Gestione di Richieste GET/POST

In Django, le viste possono essere funzioni (Function-Based Views) o classi (Class-Based Views). In una vista funzione, si differenzia GET e POST controllando `request.method`, analogo a Flask. Django fornisce strutture avanzate come le generic views che semplificano questo tipo di logica, ma comprendere l'approccio manuale è istruttivo. Per accedere ai dati di un form inviato via POST, si usa `request.POST` (un dizionario che contiene i campi inviati). Per parametri GET (querystring), si utilizza `request.GET`. Anche in Django `request.FILES` gestisce i file caricati.

Nel seguente esempio, definiamo una vista semplice che gestisce un form di input nome come nel caso Flask. Se la richiesta è GET, renderizza un template con il form; se è POST, estrae il nome inviato e restituisce una conferma. Stiamo usando `render` per GET (supponendo un template `invia_form.html` già predisposto con il form). Notare che Django applica automaticamente protezione CSRF sui form: per poter inviare con successo un form POST dal template, è necessario includere `{% csrf_token %}` all'interno del form HTML generato, altrimenti Django bloccherà la richiesta per sicurezza.

```

from django.shortcuts import render
from django.http import HttpResponse

def invia_dati(request):
    if request.method == 'POST':
        nome = request.POST.get('nome')
        # Legge il campo 'nome' dal form inviato
        return HttpResponse(f"Dati ricevuti: {nome}") # Risponde con una
        semplice conferma
    else:
        return render(request, 'invia_form.html')    # Mostra pagina con il
        form (template separato)

```

(Nel template `invia_form.html` dovrà esserci un form con `method="post"` e `{% csrf_token %}` per funzionare correttamente il POST in Django.)

Flask: Gestione dei Form con Flask-WTF

Validare manualmente i dati di un form per ogni campo può risultare ripetitivo e soggetto a errori. **Flask-WTF** è un'estensione di Flask basata su WTForms che fornisce un modo dichiarativo di definire form e validazione. Con Flask-WTF definiamo una classe form come sottoclasse di `FlaskForm`, con attributi che rappresentano i campi. Ogni campo può avere validatori (ad esempio `DataRequired` per

campi obbligatori, `Email` per formati, ecc.). Inoltre, Flask-WTF integra la protezione CSRF generando un token automaticamente incluso nel form.

Per usare Flask-WTF, bisogna installarlo (`pip install flask-wtf`) e configurare un `SECRET_KEY` nell'app per i token CSRF. Vediamo un esempio: definiremo una classe `LoginForm` con due campi (username e password) e quindi una vista di login che gestisce GET (mostra il form) e POST (valida e processa). `form.validate_on_submit()` verifica se la richiesta è POST e i campi soddisfano i validatori. In caso positivo, procederemo con l'autenticazione (qui simulata con un semplice controllo statico); altrimenti, il form riporterà gli errori da visualizzare.

```
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired

app.config['SECRET_KEY'] = 'chiave-segreta-molto-complessa'
# Necessaria per abilitare CSRF e sessioni

class LoginForm(FlaskForm):
    username = StringField('Username', validators=[DataRequired()])
    # Campo testo obbligatorio
    password = PasswordField('Password', validators=[DataRequired()])
    # Campo password obbligatorio
    submit = SubmitField('Accedi') # Pulsante di invio

@app.route('/login', methods=['GET', 'POST'])
def login():
    form = LoginForm() # Istanza il form
    if form.validate_on_submit(): # Se è un POST valido
        (passati tutti i validator)
        user = form.username.data
        pwd = form.password.data
        # Esempio di autenticazione fittizia:
        if user == "admin" and pwd == "12345": # (In produzione,
            verificare hash password in DB!)
            # Autenticazione riuscita: salva l'utente in sessione o altro
            return "Login eseguito con successo!"
        else:
            return "Credenziali non valide", 401
    return render_template('login.html', form=form) # Per GET o per POST con
    errori, mostra di nuovo il form
```

Django: Gestione dei Form e Validazione

Django include un robusto sistema di **Forms** integrato. Si possono creare classi che estendono `forms.Form` (o `forms.ModelForm` se vogliamo collegarlo a un modello). Una classe Form definisce campi (come `CharField`, `EmailField`, ecc.) e può includere validatori o metodi di pulizia personalizzati (`clean_<field>`). Django gestisce anche la presentazione (può generare HTML per il

form) e la protezione CSRF automaticamente nei template quando si usa `render` e il form viene passato al contesto.

Vediamo un esempio di form di login con Django. Definiamo `LoginForm` con campi `username` e `password`. Nella vista, istanziamo il form con i dati POST per validarlo in caso di richiesta POST (`form = LoginForm(request.POST)`). Il metodo `form.is_valid()` esegue la validazione (campi obbligatori, lunghezze, formati, ecc., in base ai field e validator definiti). Se valido, i dati puliti sono accessibili in `form.cleaned_data`. In caso di errori, il form contiene un attributo `form.errors` con dettagli che possono essere mostrati nel template.

```
from django import forms

class LoginForm(forms.Form):
    username = forms.CharField(max_length=150, required=True)
    password = forms.CharField(widget=forms.PasswordInput, required=True)

def login_view(request):
    if request.method == 'POST':
        form = LoginForm(request.POST)          # Lega i dati POST al form
        if form.is_valid():
            user = form.cleaned_data['username']
            pwd = form.cleaned_data['password']
            # Esempio di autenticazione fittizia:
            if user == "admin" and pwd == "12345":
                return HttpResponse("Login eseguito con successo!")
            else:
                return HttpResponse("Credenziali non valide", status=401)
        else:
            form = LoginForm()                  # Form vuoto per GET
    return render(request, 'login.html', {'form': form})
```

Validazione e Messaggi di Errore nei Form

Un aspetto cruciale è mostrare all'utente messaggi di errore chiari quando la validazione fallisce (ad esempio campi obbligatori vuoti, formato email errato, ecc.). Sia Flask-WTF che Django Forms forniscono meccanismi per accedere agli errori. In Flask-WTF, `form.errors` è un dizionario con gli errori per campo. In Django, `form.errors` similmente contiene errori per campo, e `form.non_field_errors` per errori generali (non associati a un singolo campo, ad esempio fallimento autenticazione). Nei template, possiamo iterare e visualizzare tali errori.

In Django, i form hanno metodi convenienti come `{{ form.as_p }}` che rende il form automaticamente con i campi avvolti in `<p>` includendo anche i messaggi di errore vicino ai campi. In alternativa, possiamo personalizzare: ad esempio mostrare errori globali in un blocco separato. Nell'esempio di template seguente, utilizziamo un form Django passato al contesto. Inseriamo il token CSRF, rendiamo il form e poi, se ci sono errori, li elenchiamo in cima. Flask-WTF è simile: bisogna inserire manualmente `form.hidden_tag()` per il token CSRF, e i campi con eventuali messaggi dal `form.errors`.

```

<!-- Template: login.html (Django) -->
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}                                <!-- Genera automaticamente i campi con
eventuali <ul class="errorlist"> -->
    {% if form.non_field_errors %}
        <div class="errors">
            {{ form.non_field_errors }}
        <!-- Errori generali non legati a singoli campi -->
        </div>
    {% endif %}
    <button type="submit">Login</button>
</form>

<!-- Note: Flask-WTF analogo, ma con sintassi leggermente diversa per
mostrare errori -->

```

Flask: Upload di File

Il caricamento di file è una parte comune dei form web (es. upload di immagini, PDF, ecc.). Flask facilita la gestione dei file caricati attraverso l'attributo `request.files`, che contiene un dizionario di oggetti file inviati dal client (richiede `enctype="multipart/form-data"` nel form HTML). Ogni file può essere ottenuto come `file = request.files['nome_campo']`. Prima di salvarlo, è buona pratica validarne il nome o tipo per evitare rischi (ad esempio, evitare nomi con path per non sovrascrivere file di sistema).

Vediamo un esempio di endpoint Flask per l'upload di file. Supponiamo di avere un form con `<input type="file" name="documento">`. Nella rotta definita con `methods=['POST']`, recuperiamo il file, quindi decidiamo dove salvarlo sul server (in una directory dedicata, p.es. `uploads/`). Qui per semplicità salviamo direttamente usando `file.save()`. In un caso reale, dovremmo assicurare che la cartella esiste e generare nomi univoci o puliti. Usiamo `secure_filename` di Werkzeug per sicurezza sui nomi. Inoltre, potremmo limitare la dimensione massima caricabile configurando `app.config['MAX_CONTENT_LENGTH']`.

```

from werkzeug.utils import secure_filename

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'documento' not in request.files:
        return "Nessun file inviato", 400
    file = request.files['documento']
    if file.filename == '':
        return "File non selezionato", 400
    nome_sicuro = secure_filename(file.filename) # Sanitizza il nome del
file
    file.save(f"./uploads/{nome_sicuro}")        # Salva il file nella
cartella uploads
    return "File caricato con successo!"

```

Django: Upload di File

Django gestisce i file upload in combinazione con il sistema di form e modelli. Solitamente si definisce un campo `FileField` o `ImageField` in un modello, oppure un campo `forms.FileField` in un form. In entrambi i casi, i file caricati appaiono in `request.FILES`. Django semplifica molto la gestione: se si usa un `ModelForm`, salvarlo salva automaticamente il file sul filesystem (in `MEDIA_ROOT`) e memorizza il percorso nel campo del modello. Qui mostriamo un approccio manuale per chiarezza.

Nell'esempio, predisponiamo una vista che si aspetta un file tramite campo `file`. Per ricevere file, il form HTML deve avere `enctype="multipart/form-data"`. Nella vista, controlliamo che ci sia `request.FILES['file']`, poi possiamo salvare il file a mano. Django fornisce un metodo `File.chunks()` per evitare di caricare file molto grandi interamente in memoria. Usiamo quindi `for chunk in file.chunks(): dest.write(chunk)` per scrivere progressivamente il file su disco. In contesti reali, configureremmo `MEDIA_ROOT` e `MEDIA_URL` per servire i file caricati, e forse useremmo il modello per riferirli.

```
def upload_view(request):
    if request.method == 'POST' and request.FILES.get('file'):
        file = request.FILES['file']                # Ottiene
        # l'oggetto file dal request
        filepath = f"./uploads/{file.name}"
        with open(filepath, 'wb') as destination:    # Apre un nuovo
            # file in scrittura binaria
            for chunk in file.chunks():
                destination.write(chunk)
        # Scrive il file a pezzi (chunks) per efficienza
        return HttpResponse("File caricato con successo!")
    return render(request, 'upload_form.html')      # Per GET, mostra
    # un template con il form di upload
```

Database e ORM: Concetti di Base

Le applicazioni web persistono i dati tipicamente in un database. L'**ORM (Object-Relational Mapping)** permette di interagire col database utilizzando oggetti Python anziché scrivere query SQL manualmente. Sia Flask (tramite librerie come SQLAlchemy) che Django (che include un ORM integrato) adottano questo approccio. Con l'ORM, definisci modelli (classi Python) che corrispondono a tabelle del database; puoi creare, leggere, aggiornare, cancellare record attraverso metodi Python, e l'ORM traduce tutto in SQL dietro le quinte.

Ad esempio, invece di fare:

```
import sqlite3
conn = sqlite3.connect('db.sqlite3')
cur = conn.cursor()
cur.execute("SELECT * FROM utenti WHERE nome=?", (nome,))
risultati = cur.fetchall()
```

Con un ORM definiremo una classe `Utente` e useremo:

```
risultati = Utente.query.filter_by(nome=nome).all()
```

o in Django:

```
risultati = Utente.objects.filter(nome=nome)
```

Questo rende il codice più pulito e portabile tra diversi tipi di database. In aggiunta, l'ORM permette di definire relazioni (uno-a-molti, molti-a-molti) come attributi e navigarle in modo naturale (es.

`utente.articoli.all()`). Di seguito vedremo come configurare un database SQLite semplice per gli esempi, e come definire modelli in Flask e Django.

```
# Senza ORM: esecuzione diretta di SQL (gestione manuale, da evitare se
possibile per complessità)
import sqlite3
conn = sqlite3.connect('mio_db.sqlite')
cur = conn.cursor()
cur.execute("INSERT INTO utenti (nome) VALUES (?)", ("Mario",))
conn.commit()
cur.execute("SELECT * FROM utenti")
print(cur.fetchall())

# Con ORM (pseudo-codice generico): definizione modello e utilizzo più
intuitivo
class Utente(BaseModello):
    id = Column(Integer, primary_key=True)
    nome = Column(String)
utente = Utente(nome="Mario") # Crea un oggetto utente
session.add(utente); session.commit() # Lo salva nel DB
lista_utenti = Utente.query.all()
# Ottiene tutti gli utenti dal DB come oggetti
```

Flask: Configurazione del Database (SQLAlchemy)

In Flask l'ORM non è integrato, ma l'estensione più comune è **Flask-SQLAlchemy**, che semplifica l'uso di SQLAlchemy (popolare ORM standalone per Python) integrandolo con la struttura Flask. Per iniziare, si installa `flask_sqlalchemy` e poi si configura la stringa di connessione al database nel `app.config['SQLALCHEMY_DATABASE_URI']`. Per SQLite, ad esempio, si usa `'sqlite:///nome_database.db'`. Si crea poi un'istanza di `SQLAlchemy(app)` che gestirà la connessione e ci fornirà la classe base per i modelli (in genere disponibile come `db.Model`).

Definiamo un modello utente come esempio, con alcuni campi. Ogni campo è un'istanza di `db.Column` con un tipo (Integer, String, etc.) e opzioni come chiave primaria, unicità, possibilità di null, lunghezza massima, ecc. Dopo aver definito i modelli, tipicamente si chiama `db.create_all()` per creare le tabelle nel database la prima volta (in sviluppo; in produzione spesso si usano migrazioni). Ecco una configurazione e definizione di modello in Flask:


```

from flask_sqlalchemy import SQLAlchemy

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'    # Database
SQLite file-based
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
# Disabilita una funzionalità deprecata
db = SQLAlchemy(app)                                         # Inizializza
l'estensione SQLAlchemy

class Utente(db.Model):
    __tablename__ = 'utenti'                                # Nome tabella (opzionale, default
derivato dal nome classe)
    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True)
    password_hash = db.Column(db.String(128))              # Hash della password (buona
pratica: salvare solo l'hash)
    def __repr__(self):
        return f"<Utente {self.nome}>"

db.create_all()      # Crea le tabelle nel database (in questo caso genera
tabella 'utenti')

```

Flask: Operazioni CRUD sul Database

Una volta definiti i modelli, possiamo eseguire le operazioni CRUD (Create, Read, Update, Delete) attraverso l'ORM. Flask-SQLAlchemy fornisce un attributo di classe `query` su ogni modello per leggere i dati, e utilizza il concetto di **sessione** (`db.session`) per registrare le modifiche da salvare. Vediamo alcuni esempi base con il modello `Utente` definito in precedenza.

Create: Creare un nuovo record significa istanziare l'oggetto e aggiungerlo alla sessione, poi fare commit.

Read: Possiamo leggere tutti gli utenti con `Utente.query.all()`, filtrare con `filter_by` o `filter` (es. `Utente.query.filter_by(nome="Mario").first()` per primo matching), ecc.

Update: Si effettua semplicemente modificando gli attributi di un oggetto recuperato e poi facendo commit.

Delete: Si usa `db.session.delete(obj)` e commit.

Esempio pratico:

```

# CREATE - aggiunta di un nuovo utente
nuovo = Utente(nome="Mario Rossi", email="mario@example.com")
db.session.add(nuovo)          # Prepara l'inserimento
db.session.commit()            # Esegue l'inserimento nel DB

# READ - query di tutti gli utenti
utenti = Utente.query.all()
print([u.nome for u in utenti]) # Stampa tutti i nomi degli utenti

```

```

# READ - filtro (utenti con nome "Mario")
mario = Utente.query.filter_by(nome="Mario Rossi").first()
if mario:
    print(mario.email)

# UPDATE - modifica di un record esistente
if mario:
    mario.email = "mrossi@example.com"
    db.session.commit()          # Salva la modifica sul DB

# DELETE - rimozione di un utente
if mario:
    db.session.delete(mario)
    db.session.commit()          # Rimuove il record dal database

```

Django: Definizione del Modello (ORM)

Django include il suo ORM nativamente. I modelli Django sono definiti come classi che estendono `django.db.models.Model`. Ogni attributo della classe corrisponde a una colonna nel database, definita con un tipo field (`CharField`, `IntegerField`, `DateField`, etc.). Django crea automaticamente un campo chiave primaria se non lo specifichiamo (tipicamente un intero autoincrementale chiamato `id`). Inoltre, definendo i modelli in `models.py` e registrando l'app nell'elenco `INSTALLED_APPS` in settings, Django li riconoscerà.

Un esempio di modello utente semplificato (da non confondere col modello utente integrato di Django) potrebbe essere:

```

from django.db import models

class Utente(models.Model):
    nome = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    data_registrazione = models.DateField(auto_now_add=True) # Data
    impostata automaticamente alla creazione

    def __str__(self):
        return f"{self.id}: {self.nome}"

```

Per applicare questo modello al database, si utilizzano le migrazioni di Django: eseguendo `python manage.py makemigrations` (che crea lo script di migrazione) e poi `python manage.py migrate` (che applica la migrazione, creando la tabella `app_nome.Utente` con i campi specificati, in questo caso). Django fornisce un potente sistema di migrazioni per gestire modifiche successive al modello (aggiungere campi, modificare tipi, ecc.) in maniera sicura senza perdere dati.

Django: Queryset e Operazioni sul Database

Una volta che i modelli sono definiti e le tabelle create tramite migrazioni, possiamo usare l'ORM Django per le operazioni CRUD. Django utilizza il concetto di **QuerySet** – un insieme di oggetti che può essere filtrato, ordinato e valutato al bisogno. Ogni modello ha un manager predefinito denominato `objects` attraverso cui eseguire query. Ecco esempi equivalenti a quelli mostrati per Flask, usando il modello `Utente` definito:

Create: Creare un record è semplice come istanziare l'oggetto e chiamare `.save()`, oppure usare il convenienza `Utente.objects.create(nome="...", email="...")`.

Read: Alcuni metodi fondamentali includono `.all()`, `.filter(condizioni)`, `.get(condizione)` (quest'ultimo lancia eccezione se non trova o se multipli trovati), `.first()` (tramite slicing QuerySet `[0]` o metodo `first` su QuerySet in Django 3.2+).

Update: O modifichiamo attributi e facciamo `.save()` su quell'istanza, oppure usiamo `QuerySet.update(condizioni)`.

Delete: Esiste `.delete()` su un'istanza o su un QuerySet.

Esempio:

```
# CREATE - aggiungi un nuovo utente
utente = Utente(nome="Luigi Bianchi", email="luigi@example.com")
utente.save() # Inserisce il nuovo utente nel database

# CREATE (alternativa rapida)
Utente.objects.create(nome="Anna Verdi", email="anna@example.com")

# READ - ottieni tutti gli utenti
tutti = Utente.objects.all()
for u in tutti:
    print(u.nome)

# READ - filtra utenti con nome che inizia per "Luigi"
luigi = Utente.objects.filter(nome__startswith="Luigi")
print(luigi.count()) # Conta quanti corrispondono

# UPDATE - modifica un utente
utente.email = "luigi.bianchi@example.com"
utente.save() # Salva i cambiamenti dell'oggetto esistente

# DELETE - elimina un utente
utente.delete() # Rimuove questo utente dal DB (ritornerà una tupla col
numero di record eliminati)
```

(Ricorda: dopo definizione o modifica modelli in Django, eseguire sempre `makemigrations/migrate` per sincronizzare il DB.)

Flask: Creazione di una API REST (JSON)

Oltre a servire pagine HTML, le nostre applicazioni spesso devono fornire **API REST** che ritornano dati (tipicamente in formato JSON) per essere consumati da client web (JavaScript/AJAX) o applicazioni esterne. Flask semplifica molto la creazione di endpoint API: una funzione di vista Flask può restituire direttamente strutture dati Python (dizionari, liste) e usando `flask.jsonify` convertirle in JSON e impostare l'intestazione appropriata `application/json`.

Per esempio, supponiamo di voler esporre un endpoint `/api/saluto` che dato un nome restituisca un saluto in JSON. Possiamo scriverlo così:

```
from flask import jsonify

@app.route('/api/saluto/<nome>')
def api_saluto(nome):
    risposta = {"messaggio": f"Ciao, {nome}!"}
    return jsonify(risposta)    # jsonify converte il dict in JSON ->
    '{"messaggio": "Ciao, Mario!"}'
```

Flask inoltre permette di specificare metodi HTTP anche per le API. Possiamo avere, ad esempio, un endpoint `/api/utenti` che su GET restituisce la lista di utenti, e su POST ne crea uno nuovo. In tal caso, controlleremo `request.method` o useremo due funzioni distinte con route differenti se preferiamo. È buona pratica restituire anche i codici di stato appropriati (200 OK di default, 201 Created per creazione, 404 Not Found se risorsa mancante, ecc.), cosa che `jsonify` consente tramite la sintassi `return jsonify(data), 201` ad esempio. Vediamo un endpoint semplice di sola lettura:

```
@app.route('/api/utenti', methods=['GET'])
def lista_utenti():
    utenti = Utente.query.all()                # Recupera tutti gli
    utenti (da SQLAlchemy)
    risultato = [{"id": u.id, "nome": u.nome} for u in utenti]
    return jsonify(risultato)                  # Ritorna una lista di
    dizionari come JSON
```

Flask: API REST con Flask-RESTful

Man mano che le API crescono, potrebbe diventare complesso gestire tutto con funzioni di vista singole. L'estensione **Flask-RESTful** fornisce un'astrazione orientata alle risorse per costruire API REST in Flask in modo ordinato. Con Flask-RESTful si definiscono classi risorsa che ereditano da `Resource` e implementano metodi corrispondenti ai verbi HTTP (get, post, put, delete). Queste risorse vengono poi aggiunte all'app tramite un oggetto `Api`, indicando la rotta.

Vediamo un esempio: creiamo un'API per gestire la risorsa Utente. Definiamo `UtenteAPI` come `Resource` con un metodo GET che restituisce i dati di un utente specifico dato l'ID, e magari un metodo DELETE per eliminarlo (solo come esempio, non implementiamo qui DELETE effettivo su DB per brevità). Poi registriamo la risorsa su una URL contenente `<int:user_id>` come parametro. Questo ci permette di avere un design RESTful pulito, es. GET `/api/utenti/42` restituisce utente 42.

```

from flask_restful import Resource, Api

api = Api(app) # inizializza Flask-RESTful API

class UtenteAPI(Resource):
    def get(self, user_id):
        utente = Utente.query.get(user_id) # Recupera utente con
        SQLAlchemy
        if utente:
            return {"id": utente.id, "nome": utente.nome, "email":
            utente.email}
        return {"errore": "Utente non trovato"}, 404

    def delete(self, user_id):
        # (Esempio: qui si potrebbe eliminare l'utente dal DB)
        return {"status": "utente eliminato"}, 204

# Aggiunge la risorsa al routing Flask-RESTful
api.add_resource(UtenteAPI, '/api/utenti/<int:user_id>')
# Flask-RESTful si occupa di collegare /api/utenti/123 a UtenteAPI.get(123)
per GET, delete() per DELETE, etc.

```

Django: API REST con DRF (Serializzazione)

In Django, per creare API REST robuste si usa comunemente **Django REST Framework (DRF)**, una libreria potente che facilita la costruzione di API e la serializzazione dei dati. DRF introduce il concetto di *serializer*, che converte i modelli Django (o altri dati) in formati come JSON (e viceversa per input). Introduce anche viste speciali (APIView, ViewSet) e router per generare automaticamente gli URL.

Per illustrare, costruiamo un'API per gli utenti definendo un serializer e una viewset. Un **ModelSerializer** è una classe che, data un modello, definisce come serializzarne i campi. Nel nostro esempio, creiamo `UtenteSerializer` che serializza il modello `Utente` includendo id, nome, email. Dopodiché, definiamo una **ViewSet** – in particolare un `ModelViewSet` – per gestire in modo automatico le operazioni CRUD su `Utente`: DRF fornisce già implementazioni di get (list/retrieve), post (create), put/patch (update) e delete (destroy) se usiamo `ModelViewSet`.

```

from rest_framework import serializers, viewsets

class UtenteSerializer(serializers.ModelSerializer):
    class Meta:
        model = Utente
        fields = ['id', 'nome', 'email'] # Specifica i campi da esporre
        nell'API

class UtenteViewSet(viewsets.ModelViewSet):
    queryset = Utente.objects.all() # Definisce l'insieme di dati
    (tutti gli utenti)

```

```
serializer_class = UtenteSerializer
# Specifica quale serializer usare per questo set di dati
```

Questa ViewSet, una volta collegata agli URL, fornirà automaticamente endpoint come: - GET `/utenti/` per listare tutti, - POST `/utenti/` per crearne uno nuovo, - GET `/utenti/<id>/` per ottenere un singolo utente, - PUT/PATCH `/utenti/<id>/` per aggiornare, - DELETE `/utenti/<id>/` per eliminare.

DRF genererà anche risposte HTTP appropriate e, se configurato, una navigazione Web browsable per testare l'API. È un enorme vantaggio in termini di velocità di sviluppo.

Django: API REST con DRF (ViewSet e Router)

Dopo aver definito un ViewSet come `UtenteViewSet`, bisogna esporlo tramite URL. Django REST Framework fornisce dei **router** che generano automaticamente le route REST standard per un ViewSet. Ad esempio, `DefaultRouter` registrato con un ViewSet creerà gli URL per list, detail, etc., come menzionato. Questo ci evita di scrivere a mano le singole url per GET, POST, ecc. Possiamo semplicemente fare:

```
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register(r'utenti', UtenteViewSet) # Registra la ViewSet
"UtenteViewSet" all'URL base "utenti"
urlpatterns = router.urls
# Include automaticamente tutte le URL generate dal router
```

Inserendo queste linee nel file `urls.py` appropriato (ad esempio `api/urls.py` incluso nel progetto), otteniamo le seguenti rotte: - GET `/utenti/` -> lista utenti (`UtenteViewSet.list` method) - POST `/utenti/` -> crea utente (`UtenteViewSet.create`) - GET `/utenti/{id}/` -> dettaglio utente (`UtenteViewSet.retrieve`) - PUT/PATCH `/utenti/{id}/` -> aggiorna utente (`UtenteViewSet.update`/ `partial_update`) - DELETE `/utenti/{id}/` -> elimina utente (`UtenteViewSet.destroy`)

Il router si occupa anche di generare nomi per queste URL (per reverse). Con DRF, aggiungere autenticazione, paginazione, permessi è questione di configurazione e poche righe di codice, mostrando la potenza del framework rispetto a implementare manualmente tutte le logiche.

```
# file: myproject/urls.py (inclusione delle API)
from django.urls import path, include
from myapp.api_urls import router # assumendo il router definito in un
modulo separato api_urls
urlpatterns = [
    # ... altre URL del sito ...
    path('api/', include(router.urls)), # Monta tutte le URL del router
    sotto il prefisso /api/
]
```

(Ora l'API Utenti sarà raggiungibile sotto `/api/utenti/` e relative sub-URL.)

Flask: Implementazione del Login

Gestire l'**autenticazione** significa permettere agli utenti di accedere con credenziali (login) e mantenere la loro sessione autenticata. In Flask abbiamo due opzioni: implementare manualmente usando le sessioni, oppure utilizzare l'estensione **Flask-Login** che fornisce un sistema completo. Qui mostreremo un approccio manuale semplice per capire il meccanismo, tenendo presente però che in un progetto reale Flask-Login offre utili funzionalità (gestione remember me, protezione route con decorator, ecc.).

Per impostazione predefinita, Flask ha un oggetto `session` (che funziona come un dizionario) per memorizzare dati lato server-client attraverso cookie firmati. Possiamo salvare l'ID utente nella sessione dopo il login. Nell'esempio seguente, consideriamo di avere un modello `Utente` con campi `username` e `password_hash`. Nel route `/login`, se la richiesta è POST, cerchiamo l'utente per username nel database e verifichiamo la password hash confrontandola con quella salvata (utilizziamo `werkzeug.security.check_password_hash` per comparare in modo sicuro). Se credenziali ok, salviamo `user_id` nella sessione e reindirizziamo al profilo; altrimenti, ritorniamo errore o ricarichiamo la pagina con un messaggio.

```
from flask import request, redirect, session
from werkzeug.security import check_password_hash

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        utente = Utente.query.filter_by(nome=username).first()    # Cerca
        utente nel DB (SQLAlchemy)
        if utente and check_password_hash(utente.password_hash, password):
            session['user_id'] = utente.id    # Salva l'ID utente in
            sessione per tenerlo loggato
            return redirect('/profilo')
        else:
            return "Credenziali non valide", 401
    # Se GET, mostra pagina di login (template con form di autenticazione)
    return render_template('login.html')
```

(Nota: È fondamentale che le password nel database siano salvate in forma hashata, mai in chiaro. In questo esempio assumiamo che `utente.password_hash` contenga un hash generato al momento della registrazione.)

Flask: Protezione delle Pagine Autenticate

Dopo aver effettuato il login, vorremo limitare l'accesso a certe rotte solo agli utenti autenticati. Nel nostro approccio manuale, ciò significa controllare la presenza di un certo valore in `session` (ad esempio `user_id`). Se non presente, l'utente non è loggato e va reindirizzato alla pagina di login. Se presente, possiamo recuperare l'utente corrente e procedere. Flask-Login semplifica questa logica fornendo un decorator `@login_required` e gestendo sessioni in background, ma vediamo come farlo manualmente qui.

Implementiamo una rotta `/profilo` accessibile solo se l'utente è in sessione. Effettuiamo il check e, in caso negativo, facciamo un redirect al login. Inoltre, prevediamo una rotta `/logout` per terminare la sessione, che si limita a fare `session.pop('user_id')` e magari un redirect alla home. Questo approccio manuale è funzionale ma deve essere ripetuto su ogni vista protetta (da qui l'utilità di Flask-Login). Comunque, per poche pagine, è sufficiente.

```
@app.route('/profilo')
def profilo():
    if 'user_id' not in session:
        return redirect('/login')          # Se non loggato, rimanda
    al form di login
    utente_id = session['user_id']
    utente = Utente.query.get(utente_id)   # Recupera i dettagli
    utente dal DB
    return f"Profilo di {utente.nome} (<a href='/logout'>Logout</a>)"

@app.route('/logout')
def logout():
    session.pop('user_id', None)          # Rimuove l'utente dalla
    sessione se presente
    return redirect('/')                  # Torna alla homepage o
    pagina di login
```

Django: Login e Autenticazione Utente

Django include un sistema di autenticazione completo: modello Utente incorporato, gestione password hash, sistema di sessioni e funzioni helper. Per eseguire un login manualmente in Django, si può usare `django.contrib.auth.authenticate()` per verificare le credenziali e `django.contrib.auth.login()` per salvare l'utente nella sessione. Queste funzioni sfruttano il modello User e il backend di autenticazione configurato (di default controlla username/password del modello User).

Nell'esempio di una vista login, usiamo il form di login precedentemente definito (`LoginForm`) oppure potremmo usare direttamente `request.POST['username']` e `request.POST['password']`. Dopo aver verificato `form.is_valid()`, chiamiamo `authenticate` passando `request`, `username` e `password`. Se restituisce un oggetto utente (significa credenziali corrette), allora usiamo `login(request, user)` per impostare la sessione. Altrimenti aggiungiamo un errore. Django gestisce la creazione del cookie di sessione e l'associazione con l'utente loggato. Da notare che il modello User di Django di default usa campi `username` e `password` (il password è hashed internamente). Ecco come potrebbe apparire:

```
from django.contrib.auth import authenticate, login as auth_login

def login_view(request):
    if request.method == 'POST':
        form = LoginForm(request.POST)
        if form.is_valid():
            username = form.cleaned_data['username']
```



```

        password = form.cleaned_data['password']
        user = authenticate(request, username=username,
password=password)
        if user is not None:
            auth_login(request, user)                # Salva lo
            stato di login nella sessione
            return redirect('profilo')
        else:
            form.add_error(None, "Credenziali non valide") # Aggiunge
            errore generale al form
        else:
            form = LoginForm()
        return render(request, 'login.html', {'form': form})

```

(In questo snippet, `login.html` dovrebbe mostrare eventuali errori generati. Stiamo utilizzando la funzione Django `login` rinominandola come `auth_login` per non confonderla col nome della vista.)

Django: Pagine Protette e Logout

Per limitare l'accesso alle pagine protette in Django, possiamo utilizzare il decoratore `@login_required` sulle viste. Questo decoratore (da `django.contrib.auth.decorators`) verifica automaticamente `request.user.is_authenticated` e in caso negativo reindirige alla pagina di login configurata (impostazione `LOGIN_URL`). Alternativamente, all'interno della vista possiamo controllare `if not request.user.is_authenticated:` manualmente. Una volta loggato, Django rende disponibile `request.user` come l'utente corrente.

Inoltre, Django fornisce la vista `django.contrib.auth.logout` (o la funzione `logout(request)`) per terminare la sessione utente. In pratica, `logout(request)` svuota i dati di sessione corrispondenti all'utente. Ecco un esempio di implementazione di pagina profilo protetta e logout:

```

from django.contrib.auth.decorators import login_required
from django.contrib.auth import logout

@login_required
def profilo(request):
    # A questo punto, sappiamo che request.user è autenticato
    return HttpResponseRedirect(f"Profilo di {request.user.username} (<a href='/logout'>Logout</a>)")

def logout_view(request):
    logout(request)                # Termina la sessione (il request.user
    diventerà anonimo)
    return redirect('login')

```

Se usiamo il decoratore `@login_required`, e l'utente non è loggato, Django lo reindirizzerà automaticamente. Per funzionare correttamente, bisogna aver impostato nella configurazione `LOGIN_URL` (default `/accounts/login/`) o passare un argomento al decoratore come `@login_required(login_url='/login/')` se la nostra pagina di login è custom.

In contesti reali, spesso si utilizzano le viste di autenticazione offerte da Django (LoginView, LogoutView) per ridurre il codice personalizzato, ma qui abbiamo mostrato il concetto manualmente.

Flask: Chiamate AJAX (JSON)

AJAX (Asynchronous JavaScript and XML/JSON) consente al front-end di comunicare col server in background, senza ricaricare tutta la pagina. In pratica oggi si utilizzano chiamate fetch/XHR che scambiano dati in formato JSON. Abbiamo già visto come creare API JSON con Flask; ora vediamo come integrarle dal lato client. Da Flask, una vista può rilevare se la richiesta è AJAX tramite intestazioni (es. `X-Requested-With`), ma spesso non serve: basta predisporre endpoint che restituiscono JSON e chiamarli via JavaScript.

Consideriamo un semplice scenario: in una pagina web Flask, abbiamo uno script JavaScript che al click di un bottone fa una richiesta GET a `/api/dati` per ottenere del contenuto JSON e poi lo mostra in pagina. Nel server Flask, predisponiamo `/api/dati` per ritornare, ad esempio, un numero casuale o un messaggio. Ecco come appare il lato Flask e un esempio lato client (JavaScript) commentato:

```
@app.route('/api/dati')
def api_dati():
    import random
    valore = random.randint(1, 100)
    return jsonify({"valore": valore})  # Ritorna un JSON del tipo
{"valore": 42}

# In un template Flask potremmo avere:
# <button id="btn">Ottieni Valore</button>
# <div id="result"></div>
# <script>
# document.getElementById('btn').addEventListener('click', function() {
#     fetch('/api/dati')
#     .then(response => response.json())
#     .then(data => {
#         document.getElementById('result').innerText = "Valore ricevuto: " +
data.valore;
#     });
# });
# </script>
```

In questo esempio, quando l'utente clicca il bottone, lo script esegue una GET a `/api/dati`. Flask risponde con JSON contenente un valore casuale. Lo script, usando la Promise `response.json()`, ottiene l'oggetto data e aggiorna il DOM mostrando il valore ricevuto. Questo principio è alla base di funzionalità dinamiche sulle pagine (aggiornare parzialmente contenuti, validare campi in background, ecc.).

Django: Chiamate AJAX (JSON)

Anche con Django possiamo facilmente creare endpoint che restituiscono JSON per essere consumati via AJAX. Possiamo scrivere una vista che utilizza `JsonResponse` (fornita da `django.http`) per restituire un dizionario come JSON. `JsonResponse` funziona in modo simile a Flask `jsonify`,

gestendo conversione e content type. Inoltre, se gli oggetti non sono serializzabili di default (ad esempio QuerySet), si possono convertire in list o dict prima.

Per esempio, realizziamo un endpoint Django che fornisce un semplice servizio (diciamo il numero di utenti registrati). Creiamo la vista `/api/numero_utenti` che conta gli utenti e risponde con un JSON. Lato client, uno script può chiamare quell'endpoint e aggiornare la pagina. Vediamo la parte server e uno snippet client:

```
from django.http import JsonResponse

def numero_utenti(request):
    count = Utente.objects.count()
    return JsonResponse({"count": count})

# Esempio lato client (in un template Django):
# <script>
# async function getUserCount() {
#   const response = await fetch("/api/numero_utenti");
#   const data = await response.json();
#   document.getElementById('userCount').innerText = `Utenti registrati: $
# {data.count}`;
# }
# getUserCount();
# </script>
# <div id="userCount"></div>
```

In questo scenario, quando la pagina carica, la funzione JavaScript `getUserCount()` fa una richiesta al server; Django risponde con qualcosa come `{"count": 42}`, e lo script inserisce il numero nel div con id `userCount`. Django REST Framework offre anche views e mixin per gestire JSON, ma per casi semplici come questo non è necessario. Ricordiamo di assicurare la protezione CSRF: le chiamate AJAX GET normalmente sono permesse, mentre per POST/PUT/DELETE via fetch dobbiamo includere il token CSRF nell'header o disabilitare temporaneamente la verifica per quell'endpoint (non consigliato in produzione).

Gestione dei File Statici

Le **risorse statiche** (CSS, JavaScript, immagini) vanno gestite in modo appropriato. In sviluppo, sia Flask che Django possono servire i file statici facilmente, ma in produzione spesso sono serviti direttamente da un web server (o CDN) per efficienza. In Flask, la cartella `static/` (di default nella directory dell'app) è dedicata a questi file. Flask rende automaticamente disponibili questi file all'URL `/static/<filename>`. Ad esempio, un file `static/style.css` sarà raggiungibile a `http://<host>/static/style.css`. Nei template Flask, si utilizza `url_for('static', filename='style.css')` per ottenere l'URL corretto.

In Django, si definisce in settings un `STATIC_URL` (di default `/static/`) e eventualmente un percorso `STATICFILES_DIRS` o cartelle static in ciascuna app. In debug mode, Django può servire questi file, ma in produzione è necessario eseguire `collectstatic` per raccogliarli in `STATIC_ROOT` e farli servire dal web server. Nei template Django, si usa `{% load static %}` e poi il tag

`{% static 'path/to/file.css' %}` per ottenere l'URL. Ecco esempi di utilizzo in entrambi i framework:

```
<!-- In un template Flask -->
<link rel="stylesheet" href="{{ url_for('static', filename='css/
style.css') }}">


<!-- In un template Django -->
{% load static %}
<link rel="stylesheet" href="{% static 'css/style.css' %}">

```

È importante assicurarsi che il percorso indicato corrisponda al file esistente nelle directory static configurate. Inoltre, per migliorare le performance, si possono usare versioni minificate dei file o pipeline di compressione, ma questo esula dagli scopi essenziali. L'importante è ricordare l'uso di `url_for('static', ...)` e `{% static %}` invece di hardcodare percorsi, così come configurare correttamente la raccolta di static in Django quando si fa deploy.

Flask: Organizzazione con Blueprint

Quando l'app Flask cresce, diventa difficile tenere tutte le rotte e logica in un unico file. I **Blueprint** di Flask offrono un modo per organizzare il codice in moduli logici riutilizzabili. Un Blueprint rappresenta un insieme di rotte, template e statici che possono essere registrati nell'app principale. Ad esempio, potremmo avere un blueprint `auth` per tutte le rotte di autenticazione, uno `admin` per le rotte di amministrazione, ecc. Ogni blueprint può avere un prefisso URL (es. tutte le rotte admin iniziano con `/admin`).

Dopo aver definito il blueprint, va registrato con `app.register_blueprint()`. Questo meccanismo consente di sviluppare componenti dell'app quasi indipendentemente e poi integrarli facilmente. Vediamo un esempio: creiamo un blueprint per le funzionalità di amministrazione.

```
from flask import Blueprint

admin_bp = Blueprint('admin', __name__, url_prefix='/admin')

@admin_bp.route('/dashboard')
def admin_dashboard():
    return "Pannello di Controllo Admin"

@admin_bp.route('/utenti')
def lista_utenti_admin():
    # (Esempio: listare utenti, richiederebbe controllo admin reale)
    return "Lista utenti per amministrazione"

app.register_blueprint(admin_bp)
```

In questo esempio, qualsiasi rotta definita in `admin_bp` avrà automaticamente il prefisso `/admin/`. Quindi la `admin_dashboard` sarà raggiungibile su `/admin/dashboard`. Possiamo separare blueprint in file diversi (tipicamente un blueprint per file o pacchetto), mantenendo il codice organizzato. Blueprint facilita anche riusare codice: un blueprint potrebbe essere reso come un pacchetto installabile e riutilizzato in più progetti Flask.

Django: Struttura Modulare (Apps)

Django incoraggia da sempre la struttura modulare attraverso le **applicazioni Django**. Ogni *app* è pensata per fornire un insieme di funzionalità (es. blog, forum, account utente) e può essere facilmente inserita o rimossa da un progetto, oppure riutilizzata in progetti diversi. Abbiamo visto che un progetto Django può contenere molte app. Questa modularità è supportata dal sistema di routing tramite `include()`, dall'isolamento di template e statici per app, e dalla registrazione dei modelli per app.

Ad esempio, potremmo avere un'app `blog` con sue viste, modelli e url. Nel `urls.py` del progetto, includeremo gli URL del blog sotto un certo prefisso (oppure root). Lo facciamo così:

```
# file: myproject/urls.py (principale del progetto)
from django.urls import path, include
urlpatterns = [
    path('blog/', include('blog.urls')),      # Tutte le url definite in blog/
    # urls.py sotto path "blog/"
    path('accounts/', include('accounts.urls')), # Esempio di seconda app
    # per gestione account
    path('', include('mainapp.urls')),         # App principale per homepage e
    # altro, montata sulla root
]
```

Ogni app registra le proprie view e route. Inoltre in `settings.py` c'è `INSTALLED_APPS` dove elenchiamo le app attive. Questo consente a Django di caricarne i modelli, template, segnali, ecc. Quando un'app non è più necessaria, si può rimuovere (tenendo presente di gestire le migrazioni di conseguenza). Questa architettura modulare rende Django molto mantenibile su grandi codebase. In sintesi: organizzare il codice in app è la "Blueprint way" di Django, con il vantaggio che Django la supporta nativamente e molte app terze parti (es. `django-allauth`, `django-rest-framework`) si integrano semplicemente aggiungendole a `INSTALLED_APPS` e includendo i loro URLs.

```
INSTALLED_APPS = [
    # ... app di sistema ...
    'blog.apps.BlogConfig',      # Registrazione app Blog (BlogConfig
    # definito in blog/apps.py)
    'accounts.apps.AccountsConfig',
    'mainapp.apps.MainappConfig',
]
# Ogni app avrà tipicamente file separati: models.py, views.py, urls.py,
# templates/blog/*.html, static/blog/*, etc.
```

Preparazione al Deploy (Configurazione)

Finora abbiamo lavorato in ambiente di sviluppo (debug mode, server integrato). **Deployment** significa prendere la nostra applicazione e metterla in esecuzione su un server accessibile agli utenti finali. Ci sono differenze chiave tra sviluppo e produzione: - Disabilitare DEBUG (in Django `DEBUG=False`, in Flask non usare debug mode). - Impostare correttamente gli *host consentiti* (Django: `ALLOWED_HOSTS`). - Utilizzare un server WSGI di produzione (es. Gunicorn) invece del server integrato. - Gestire variabili d'ambiente per configurazioni sensibili (chiavi segrete, credenziali DB, ecc.).

Prima del deploy, quindi, si: 1. Crea un file di **requirements.txt** con le dipendenze (`pip freeze > requirements.txt`). 2. Configura parametri come SECRET_KEY, DATABASE_URL, ecc. per l'ambiente di produzione, preferibilmente tramite variabili d'ambiente piuttosto che hardcodate. 3. In Django, raccogli i file statici con `manage.py collectstatic`. 4. Prepara un file WSGI/ASGI (per Flask è generalmente l'oggetto app stesso, per Django c'è `project/wsgi.py`). 5. Testa in locale con un server come Gunicorn: ad esempio `gunicorn app:app` per Flask, o `gunicorn myproject.wsgi:application` per Django, per assicurarsi che parta correttamente.

Esempio di impostazioni usando variabili d'ambiente (in Python, tramite `os.environ`):

```
# Esempio: in config Flask o settings Django
import os
DEBUG = os.environ.get('DEBUG', 'False') == 'True'
SECRET_KEY = os.environ.get('SECRET_KEY', 'sostituisci_questa_chiave')
# In Django:
ALLOWED_HOSTS = os.environ.get('ALLOWED_HOSTS', 'localhost').split(',') #
es. "miosito.pythonanywhere.com"
```

In pratica, su servizi come PythonAnywhere o Render, queste variabili si settano nel pannello di controllo. Utilizzarle rende il nostro codice più sicuro (non contiene segreti) e flessibile per diversi ambienti.

Deploy di Flask su PythonAnywhere

PythonAnywhere è un hosting Python in cloud che offre un piano gratuito per piccole app. Fare deploy su PythonAnywhere di una app Flask richiede pochi passi: - Registrare un account e creare una nuova **Web App** selezionando Flask e la versione di Python appropriata. - PythonAnywhere configurerà un ambiente WSGI e chiederà il percorso dell'app (ad esempio `~/mysite/app.py`). - Caricare i file dell'applicazione sul server (via git, SFTP o editor integrato). - Installare le dipendenze (c'è una console bash sul sito per eseguire `pip install -r requirements.txt` in un virtualenv). - Configurare la WSGI file (PythonAnywhere genera un file e.g. `var/www/mysite_wsgi.py`): in questo file bisogna importare l'app Flask.

Un esempio minimal di file WSGI per Flask su PythonAnywhere:

```
# file: /var/www/YourUsername_pythonanywhere_com_wsgi.py
import sys
# Aggiunge il percorso del tuo progetto al PYTHONPATH
path = '/home/YourUsername/mysite'
```

```

if path not in sys.path:
    sys.path.append(path)

# Imposta la variabile d'ambiente FLASK_APP se necessario (non strettamente
# richiesta qui)
# os.environ['FLASK_APP'] = 'app'

# Importa l'app Flask dal tuo file app.py
from app import app as application # 'app' è l'istanza Flask nel tuo app.py

```

Dopo aver configurato, si può fare **Reload** dell'app dal dashboard PythonAnywhere. Se tutto è corretto, la tua app Flask sarà live all'URL `YourUsername.pythonanywhere.com`. Ricorda di impostare correttamente eventuali variabili (es. `SECRET_KEY`) e di disabilitare debug. PythonAnywhere ha anche log accesso/errori visualizzabili per debug in caso di problemi.

Deploy di Django su PythonAnywhere

Distribuire un progetto Django su PythonAnywhere è simile nei passi iniziali: - Creare nuova Web App, scegliendo Django e la versione Python. PythonAnywhere chiederà alcune info come il percorso del `manage.py` e creerà un file WSGI precompilato per Django. - Caricare il progetto via git/SFTP. - Installare le dipendenze nel virtualenv (`requirements.txt`). - **Configurare settings di produzione:** impostare `DEBUG=False`, aggiungere il dominio `<username>.pythonanywhere.com` in `ALLOWED_HOSTS`, e assicurarsi di avere un `SECRET_KEY` forte (PythonAnywhere ne genera uno se si è usato il loro starter). - Eseguire le migrazioni sul database di produzione (ad esempio SQLite sul filesystem o MySQL fornito dal servizio) con `manage.py migrate` e creare un superuser se necessario.

Il file WSGI per Django su PythonAnywhere (`mysite_pythonanywhere_com_wsgi.py`) in genere includerà:

```

import os
import sys

path = '/home/YourUsername/mysite' # path al tuo progetto
if path not in sys.path:
    sys.path.append(path)

os.environ['DJANGO_SETTINGS_MODULE'] = 'myproject.settings' # indicare il
modulo settings del progetto
from django.core.wsgi import get_wsgi_application
application = get_wsgi_application()

```

Django su PythonAnywhere funzionerà all'URL di default fornito. Per i file statici, su PythonAnywhere conviene usare il meccanismo di **Static files** nel tab Web app: si può mappare l'URL `/static/` a la directory `~/mysite/static` (dopo aver eseguito `collectstatic`). In questo modo, PythonAnywhere servirà direttamente i file statici. Dopo setup, cliccare Reload. L'app Django dovrebbe ora essere accessibile. Eventuali errori di configurazione appariranno nei log (error log).

Deploy di Flask su Render.com

Render.com è un moderno servizio di hosting (simile a Heroku) che offre un tier gratuito. Per Flask, un modo semplice di deploy è: - Avere il codice in un repository Git (ad esempio GitHub). - Creare un nuovo servizio Web su Render collegando il repository. - Specificare come **Start Command** qualcosa tipo: `gunicorn app:app` (se `app.py` contiene l'istanza Flask chiamata `app`). - Render rileva automaticamente un'app Python se c'è un `requirements.txt` e installerà le dipendenze. - Configurare le variabili d'ambiente nel dashboard (es. `FLASK_ENV=production`, `SECRET_KEY=<...>` etc.).

Bisogna includere Gunicorn nei requirements perché Render lo userà per l'esecuzione. Un file `Procfile` può essere usato per Heroku/Render per specificare il comando di avvio. Ad esempio, un Procfile per Flask potrebbe contenere:

```
web: gunicorn app:app --preload
```

Quando si push il codice, Render effettua il build e avvia il servizio. Lo `--preload` è facoltativo (carica l'app all'avvio, utile per errori di import visibili subito). Assicurarsi che `app:app` punti correttamente (Formato `file:istanza_flask`). Nel nostro caso, se in `app.py` abbiamo `app = Flask(__name__)`, allora `app:app` va bene.

Render fornisce un URL del tipo `mio-progetto.onrender.com`. Possiamo testare dopo il deploy se l'app risponde alle richieste. Se ci sono problemi, Render mostra i log (build log e runtime log). Un tip: disattivare il debug mode come sempre, e magari configurare logging per debug.

Deploy di Django su Render.com

Per Django su Render, il processo è simile ma richiede qualche passo aggiuntivo: - Aggiungere Gunicorn e maybe Whitenoise (per static files) a `requirements.txt`. - Creare un file `Procfile` con: `web: gunicorn myproject.wsgi:application --preload` per avviare Django. - Configurare le variabili d'ambiente su Render: ad esempio `DJANGO_SECRET_KEY`, `DJANGO_DEBUG=False`, `DATABASE_URL` se si usa un database esterno (Render può fornire Postgres). - Render quando effettua il build, se rileva migrazioni, è bene eseguire `manage.py migrate`. Render permette comandi di migrazione come parte del deploy (c'è uno setting di "migration command").

Per i file statici in Django su Render, una soluzione semplice è usare **WhiteNoise**, una libreria che consente a Django di servire static direttamente. Installando whitenoise e aggiungendo in `settings.py`:

```
MIDDLEWARE = ['whitenoise.middleware.WhiteNoiseMiddleware', ...] # come
primo middleware
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles') # dove collectstatic
raccoglie
```

E poi eseguire `python manage.py collectstatic` durante il build (Render permette di configurare un comando post-build). Così i static files saranno inclusi nell'app e serviti da whitenoise.

Infine, come con Flask, configurare in Render il deploy da Git. Una volta deployato, l'app Django sarà disponibile ad un URL dedicato. Ricordare di aggiungere quel URL ai `ALLOWED_HOSTS`. Su Render, questo può essere fatto fix nel settings o passato come env var, e in settings fare:

```
ALLOWED_HOSTS = os.environ.get('ALLOWED_HOSTS', '').split(',')
```

e poi su Render mettere l'host.

Sicurezza Web: XSS e CSRF

Durante lo sviluppo web, bisogna considerare la sicurezza. Due minacce comuni lato applicazione sono **XSS (Cross-Site Scripting)** e **CSRF (Cross-Site Request Forgery)**. Fortunatamente, sia Flask che Django forniscono protezioni di base: - **XSS**: entrambi i motori di template (Jinja2 e Django) eseguono di default l'escaping di caratteri HTML per variabili inserite nei template. Ciò significa che se un utente malintenzionato inserisce `<script>alert('hack')</script>` in un campo, quando lo mostreremo con `{{ campo }}` verrà trasformato in stringa innocua (`<script>...`). Bisogna però stare attenti quando si marcano stringhe come "sicure" (es. in Django `|safe` filter), perché si disabilita l'escaping. - **CSRF**: Django ha integrato un sistema di token CSRF. Ogni form HTML deve includere `{% csrf_token %}` e Django verifica che ogni richiesta POST contenga un token valido di sessione. Flask, tramite Flask-WTF, offre un meccanismo simile: ogni form generato avrà un campo nascosto con token e valida automaticamente su `validate_on_submit`. Se non si usa Flask-WTF, bisogna implementare CSRF a mano o tramite estensioni.

Dimostriamo con un piccolo esempio l'effetto dell'escaping (contro XSS) usando Werkzeug/markupsafe manualmente, per capire cosa succede:

```
from markupsafe import escape
user_input = "<b>Hello</b> <script>alert('XSS')</script>"
print(escape(user_input))
# Output: &lt;b&gt;Hello&lt;/b&gt; &lt;script&gt;alert('XSS')&lt;/script&gt;
```

Come si vede, `<` e `>` vengono sostituiti da entità HTML. Nel browser questo non esegue nessun JavaScript, ma mostra letteralmente `<script>...`. I framework fanno questo automaticamente per variabili nei template. Per il CSRF, se provassimo a inviare un POST al nostro endpoint Django senza token, otterremmo un errore `403 Forbidden (CSRF token missing or incorrect)`. Quindi, assicurarsi di includere il token nei form (Django) o di usare l'estensione CSRFProtect in Flask se non si adopera Flask-WTF.

Gestione degli Errori (404/500)

In produzione, è importante gestire con eleganza gli errori come 404 (pagina non trovata) o 500 (errore interno). Entrambi i framework ci permettono di intercettare questi errori e definire risposte personalizzate (ad esempio pagine di errore user-friendly). - In **Flask**, possiamo usare il decoratore `@app.errorhandler(code)` per specifici codici. Ad esempio, `@app.errorhandler(404)` definisce una funzione che restituisce un messaggio o una pagina per ogni 404. Possiamo anche usare `abort(404)` nelle viste per generare un 404 manualmente. - In **Django**, si creano template speciali chiamati `404.html` e `500.html` nella directory template root del progetto. Django li userà

automaticamente quando avvengono tali errori (con `DEBUG=False`). Inoltre, possiamo definire viste custom per errori (nel `urls.py` del progetto, parametri `handler404 = 'mymodule.my_view'`). In genere però basta creare i template di errore.

Esempio in Flask:

```
from flask import render_template, abort

@app.errorhandler(404)
def pagina_non_trovata(error):
    return render_template('404.html'), 404

# In una vista Flask potremmo utilizzare abort() se qualcosa non esiste:
@app.route('/articolo/<int:id>')
def mostra_articolo(id):
    articolo = Articolo.query.get(id)
    if articolo is None:
        abort(404) # Genera un errore 404 che sarà gestito
    dall'errorhandler
    return render_template('articolo.html', articolo=articolo)
```

Esempio in Django: - Creare `templates/404.html` con un messaggio tipo "Pagina non trovata". - Django, quando `DEBUG=False` e non trova una pagina, servirà quel template con status 404 automaticamente. - Per errori 500 (eccezioni non gestite), mettere un `500.html` con un generico messaggio di scuse.

Testare queste pagine in sviluppo può essere fatto impostando `DEBUG=False` temporaneamente e simulando le condizioni. Una buona pagina di errore tiene lo stile del sito e magari un link per tornare alla home.

Django Admin: Pannello di Amministrazione

Uno dei grandi vantaggi di Django è il **site admin** automatico. Django fornisce un'app di amministrazione generica che, una volta attivata, permette di gestire i modelli (inserire, modificare, cancellare record) tramite un'interfaccia web protetta da login. Questo è estremamente utile per popolare contenuti o fare correzioni veloci senza implementare apposite pagine. Per usare l'admin: - In `settings.py`, assicurarsi che `'django.contrib.admin'` sia in `INSTALLED_APPS` (lo è per default). - Includere gli URL dell'admin nel `urls.py` principale: `path('admin/', admin.site.urls)`. - Creare un superuser con `manage.py createsuperuser` per avere accesso.

Per rendere visibili i nostri modelli nell'admin, dobbiamo registrarli. Nel file `admin.py` della nostra app, importiamo il modello e utilizziamo `admin.site.register(Model)` (o creiamo una classe `ModelAdmin` personalizzata per più controllo). Una volta fatto, avviando il server e andando su `/admin/`, potremo loggarci e vedere i modelli registrati.

Esempio:

```
# file: myapp/admin.py
from django.contrib import admin
from .models import Utente, Articolo

admin.site.register(Utente)
admin.site.register(Articolo)
```

Possiamo personalizzare molto l'admin (campi da mostrare, filtri, ricerca, ecc.), ma anche senza personalizzazioni avremo moduli di inserimento/modifica per ogni modello. L'admin è uno strumento potentissimo durante lo sviluppo e anche in produzione per amministratori di sistema o redattori di contenuti. Bisogna assicurarsi di limitarne l'accesso (per default è protetto e occorre quell'account superuser o staff). Flask non ha un equivalente integrato, ma esistono estensioni come Flask-Admin che forniscono interfacce generiche, sebbene non così complete come l'admin di Django.

```
# Ad esempio, dopo aver creato un superuser, potremo loggarci su http://
localhost:8000/admin/
# e vedere Utente, Articolo nell'elenco, gestire i loro record con pochi
clic.
```

Flask vs Django: Differenze Chiave

Flask e Django perseguono obiettivi simili (facilitare lo sviluppo web) con approcci differenti: - **Struttura:** Django è *opinionated* – impone una struttura rigida (MTV: Model-Template-View), generando un progetto con convenzioni già pronte. Flask è *unopinionated* – ti dà un canvas vuoto da organizzare come preferisci, consigliando ma non imponendo. - **Funzionalità incluse:** Django offre tantissimo integrato: ORM, form, auth, admin, motore template, sistema di migrazioni... Flask di base include routing e template; per tutto il resto si ricorre a estensioni o librerie a scelta (che offre grande flessibilità). - **Taglia del progetto:** Per progetti grandi e complessi, Django grazie alla sua struttura modulare e funzionalità predefinite può far risparmiare tempo e mantenere l'ordine. Per progetti piccoli o microservizi, Flask è più leggero, con meno sovrastruttura, quindi risulta più indicato. - **Comunità e ecosistema:** Entrambi hanno grandi community. Django esiste da più tempo (dal 2005) e ha un ecosistema maturo di plugin (app riutilizzabili per blog, forum, ecc.). Flask, pur più giovane (2009), è popolarissimo specialmente per API e servizi, con molte estensioni (Flask-Login, Flask-RESTful, Flask-Migrate, etc.). - **Performance:** Dipende molto dall'implementazione, ma Flask essendo minimale ha un footprint leggermente minore. Django però è altamente ottimizzato e per applicazioni complesse le differenze di performance sono marginali rispetto ai benefici.

In breve: - **Flask** – Pro: semplice, flessibile, curva di apprendimento rapida se conosci Python; **Contro:** richiede scrivere o scegliere componenti per funzionalità comuni, rischio di reinventare la ruota se non si conoscono le estensioni.

- **Django** – Pro: pronto all'uso, segue best practice di sicurezza e progettazione, DRY (don't repeat yourself) con admin e form generici; **Contro:** inizialmente può sembrare macchinoso per cose semplici, meno adatto a piccoli servizi singolo scopo.

La scelta tra Flask e Django dipende dal progetto: un'app API leggera o un prototipo veloce può beneficiare della snellezza di Flask, mentre un sito data-driven con pannello admin, autenticazione complessa, ecc. probabilmente è più rapido da implementare con Django.

Testing delle Applicazioni Web

La scrittura di test automatizzati assicura che le funzionalità dell'app funzionino e continuino a funzionare dopo future modifiche. Entrambi i framework supportano bene il testing: - Flask fornisce un client di test integrato che permette di inviare richieste alla nostra app senza avviarla realmente su una porta. Usando `app.test_client()` possiamo simulare chiamate GET/POST e verificare le risposte. - Django ha un framework di test potente integrato. Possiamo creare testcases che ereditano da `django.test.TestCase`, usare un client di test (`self.client.get()`, `.post()`, ecc.) e inoltre Django gestisce un database di test isolato (applica le migrazioni su un DB temporaneo per eseguire i test).

Esempio di test in Flask (usando pytest o unittest):

```
# test_app.py
import pytest
from app import app

@pytest.fixture
def client():
    with app.test_client() as client:
        yield client

def test_homepage(client):
    resp = client.get('/')
    homepage
    assert resp.status_code == 200
    assert b"Benvenuto" in resp.data
    # verifica che contenga la parola "Benvenuto"
    # Simula una GET alla
```

Esempio di test in Django:

```
# file: myapp/tests.py
from django.test import TestCase

class SimpleTest(TestCase):
    def test_homepage(self):
        response = self.client.get('/')
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Benvenuto")
```

L'approccio è simile: si verifica lo status code e contenuto della risposta. Possiamo anche testare la logica di backend separatamente (es. funzioni, metodi dei modelli). Per Django, potremmo testare che la creazione di un modello funziona, che un form segnala errori correttamente, ecc. I test diventano cruciali man mano che l'app cresce, per evitare regressioni. Entrambi i framework permettono anche test più avanzati, come simulare login (Django ha `client.login()` per test, Flask potremmo impostare sessione test), o testare API (in Django REST Framework esiste `APIClient` per testare con facilità le API).

Risorse, Librerie e Community

Per approfondire e risolvere problemi, è fondamentale conoscere la documentazione e risorse disponibili: - **Documentazione Ufficiale Flask** – <https://flask.palletsprojects.com/> Chiara e ricca di tutorial (Flask Mega-Tutorial di Miguel Grinberg è una risorsa molto popolare). - **Documentazione Ufficiale Django** – <https://docs.djangoproject.com/> (in italiano disponibili vecchie versioni tradotte, ma la più aggiornata è in inglese). Include una guida introduttiva passo-passo eccellente. - **Community & Forum**: Stack Overflow ha migliaia di Q&A su Flask e Django; Reddit (r/flask, r/django) e Discord/Slack comunitari per sviluppatori Django e Flask sono attivi. - **Librerie Flask utili**: Flask-Login (autenticazione), Flask-WTF (form), Flask-Migrate (gestione migrazioni DB con Alembic), Flask-RESTful / Flask-RESTX (creazione API REST), Flask-Mail (inviare email). - **Librerie Django utili**: Django REST Framework (API), django-allauth (autenticazione OAuth/social), django-debug-toolbar (debug query e performance), django-extensions (utility extra), celery (per task asincroni, compatibile con Django), Channels (per WebSocket in Django). - **Esempi e progetti open source**: Consultare repository GitHub di progetti Flask/Django per vedere come sono strutturati. Ad esempio, Flaskr (app demo Flask), oppure il progetto Django di esempio integrato (blog) nella documentazione. - **Libri e corsi**: *Flask Web Development* (M. Grinberg) per Flask; *Django for Beginners* (W. Vincent) per Django, sono buoni testi se si vuole un percorso guidato.

Investire un po' di tempo nella lettura della doc ufficiale porta benefici enormi, poiché spesso framework e librerie hanno dettagli importanti (ad esempio protezioni di sicurezza) spiegati lì.

Esercizio Pratico Consigliato

Per consolidare quanto appreso, niente è meglio di un po' di pratica. Un esercizio consigliato è sviluppare una piccola applicazione sia in Flask che in Django, confrontando l'esperienza. Ad esempio, proviamo a costruire una **To-Do List** con funzionalità base: - Elenco dei task da fare, con possibilità di aggiungerne di nuovi e marcarli come completati. - In Flask: definire un modello Task (magari usando SQLAlchemy o anche una semplice lista in memoria per iniziare), rotte per visualizzare tutti i task, aggiungerne (form POST) e completarne (ad esempio con una richiesta POST/PUT). - In Django: creare un'app `tasks`, modello Task, utilizzare le generic views o crearle manualmente per lista e creazione, e magari interfaccia admin per gestirli rapidamente.

Passi suggeriti (Flask): 1. Definisci la struttura (app Flask, configura SQLAlchemy, modello Task con campi id, descrizione, completato booleano). 2. Crea una route GET `/tasks` che mostra i task (template con elenco e form per aggiungerne). 3. Crea una route POST `/tasks/add` per aggiungere un nuovo task dal form. 4. Crea una route `/tasks/complete/<id>` che segna come completo un task (possibilmente POST – es. form con metodo POST o chiamata AJAX). 5. (Opzionale) Aggiungi stile con CSS nel static e magari un po' di JS per effetto di spunta.

Passi suggeriti (Django): 1. Crea progetto e app `tasks`, aggiungila a INSTALLED_APPS. 2. Modello Task (descrizione CharField, completato BooleanField). 3. Crea una vista basata su classe ListView per mostrare i task e un CreateView per aggiungerli, oppure viste funzione equivalenti. 4. Configura urls.py dell'app e includilo nel progetto. 5. Utilizza il potente admin per aggiungere tasks o marcarli completi come prova. 6. (Opzionale) Aggiungi un'azione custom nel admin per segnare multipli task completati, giusto per esplorare.

Implementando questa semplice app in entrambi i framework, rafforzerai la comprensione di routing, template, form e database. Confronterai anche come Flask richiede manualità (ad esempio dovrai tu

gestire redirezioni dopo aggiunta, query al modello, etc.), mentre Django offre scorciatoie (le generic views e il form model). Entrambi gli approcci arricchiscono le tue abilità di sviluppatore web Python.

Altri Strumenti e Framework da Esplorare

Python offre molte altre opzioni e strumenti per lo sviluppo web oltre Flask e Django: - **FastAPI**: un framework moderno e molto performante per creare API (particolarmente REST/JSON) basato su ASGI e tipizzazione Python. È asincrono per natura e ottimo per microservizi e API veloci. Esempio minimo:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def read_root():
    return {"message": "Hello FastAPI"}
```

FastAPI è noto per la facilità con cui genera documentazione interattiva delle API (OpenAPI). - **Pyramid**: un altro framework general-purpose, configurabile e modulare, meno usato di Django ma apprezzato per la sua flessibilità. - **Tornado / Sanic / Starlette**: framework o server asincroni che supportano WebSocket e alti carichi concorrenti. - **Flask-Admin / Django Jet / etc.**: per interfacce di amministrazione personalizzate oltre l'admin base. - **Front-end integration**: potresti voler esplorare come integrare un front-end più ricco: ad esempio usare React/Vue/Angular con un backend Django o Flask (lo scenario tipico è backend con API REST e front-end single-page app). - **Docker e Containerizzazione**: per distribuire facilmente le tue app, imparare a creare un Dockerfile per Flask o Django e usare docker-compose (es. per includere un servizio database) è molto utile. - **CI/CD**: strumenti come GitHub Actions, Jenkins o GitLab CI per automatizzare test e deploy. - **Performance e scaling**: se il progetto cresce, considera caching (Django ha framework caching integrato, Flask ha estensioni), e tecniche come load balancing (servire con più worker Gunicorn), e l'uso di CDN per static.

Ognuno di questi argomenti può essere esplorato una volta consolidati i fondamentali. Ad esempio, potresti rifattorizzare la tua To-Do app Flask per usare FastAPI e confrontare; oppure provare Django Channels per aggiungere notifiche in tempo reale ai task completati (via WebSocket). Il mondo dello sviluppo web è vasto, ma la solida base Python + Flask/Django ti permette di progredire verso qualsiasi direzione con buoni fondamenti.

Riepilogo dei Concetti Chiave

Abbiamo coperto un ampio spettro di argomenti essenziali per lo sviluppo web con Python. Ripercorriamo brevemente i punti principali: - **Framework Flask e Django**: Flask è leggero e flessibile, Django strutturato e completo; la scelta dipende dai requisiti del progetto. - **Routing e URL**: definire percorsi che mappano a funzioni/viste; parametri nelle URL per contenuti dinamici; generazione di URL inversa (`url_for` in Flask, `reverse` / `{% url %}` in Django) per evitare hardcoding. - **Template engine**: utilizzare Jinja2 (Flask) o il motore Django per generare HTML dinamico; separazione logica/presentazione; ereditarietà di template per layout consistenti; safe escaping per prevenire XSS. - **HTTP e richieste**: distinguere metodi GET/POST; acquisire dati da `request` (query param, form data); restituire risposte appropriate; redirect e gestione codici di stato (ad es. 404). - **Form e validazione**: costruire form HTML e processarli server-side; usare librerie (WTForms/Flask-WTF, Django Forms) per semplificare validazione e gestione errori; protezione CSRF su form POST. - **Database e ORM**: definire modelli rappresentativi di tabelle; configurare la connessione (SQLite per test/dev); eseguire operazioni CRUD via ORM; differenze tra SQLAlchemy (usato in Flask) e Django ORM (integrato); importanza delle

migrazioni per Django. - **API REST:** creare endpoint che forniscono JSON invece di HTML; in Flask, utilizzare `jsonify` o Flask-RESTful per strutturare risorse; in Django, sfruttare Django REST Framework per rapidità e funzionalità avanzate (serializers, ViewSet, router). - **Autenticazione di base:** predisporre sistema login/logout; in Flask manipolando sessioni o usando estensioni come Flask-Login; in Django usando l'auth system integrato, login_required e il comodo admin per gestire utenti; ricordarsi di proteggere rotte riservate. - **AJAX e interazione client-server:** come effettuare chiamate asincrone dal front-end (fetch API, jQuery AJAX) per aggiornare contenuti senza reload; predisporre viste lato server che rispondano con JSON e gestire tali risposte nel front-end; considerazioni di CSRF per chiamate POST via AJAX. - **Static files e asset:** organizzare CSS/JS/immagini; usare gli strumenti del framework (`url_for('static', ...)` e `{% static %}`) per referenziarli; capire la differenza tra servire statici in sviluppo vs produzione (collectstatic in Django, static_folder in Flask). - **Blueprints/Apps (modularità):** strutturare progetti grandi dividendoli in componenti: blueprint Flask registrabili con prefissi; apps Django con loro urls, views, models integrate nel progetto. - **Deployment:** passare da ambiente locale a server live; spegnere debug, configurare host e variabili; utilizzare servizi gratuiti (PythonAnywhere per semplicità di configurazione manuale, Render per integrazione CI); preparare WSGI/ASGI, usare Gunicorn; gestire file statici su server; importanza di mantenere segreti fuori dal codice (env vars). - **Sicurezza e best practice:** difendersi da XSS con template autoescape, da CSRF con token nei form; hashing password e non conservarle in chiaro; gestire errori HTTP con pagine dedicate per migliorare UX; validare input utente sia lato client (per usabilità) che lato server (per sicurezza). - **Strumenti aggiuntivi:** come l'admin Django per risparmiare tempo su interfacce gestionali, test automated per prevenire regressi, e risorse di comunità e documentazione per continuo apprendimento.

In sintesi, padroneggiando questi concetti, sei in grado di sviluppare un'applicazione web Python completa, che va dalla semplice pagina di saluto fino a un sistema con autenticazione, database e API interattive.

Prossimi Passi

Ora che hai una panoramica dello sviluppo web essenziale con Flask e Django, il passo successivo è mettere in pratica e poi approfondire: - **Consolidare con un progetto reale:** Scegli un'idea (blog, rubrica contatti, piccolo e-commerce) e prova a implementarla in Django o Flask. Affrontare esigenze reali (upload immagini, paginazione di liste, ricerca, etc.) ti farà scoprire parti aggiuntive dei framework (es. paginatori Django, Blueprints più complessi in Flask). - **Approfondire il framework scelto:** Se preferisci Django, potresti esplorare la creazione di API con Django REST Framework o l'uso di Channels per real-time (websocket) features. Se preferisci Flask, potresti concentrarti su Flask-RESTX (un fork migliorato di Flask-RESTful) o l'integrazione di Flask con librerie frontend come React (servendo l'app React e usando Flask come API). - **Ottimizzazione e scalabilità:** Studia come gestire caching (Django ha memcache support, Flask può usare Flask-Caching), logging avanzato, gestione di errori con servizi come Sentry, e come eseguire l'app in ambienti container (Docker) per deployment ripetibili. - **Sicurezza avanzata:** Implementa autenticazione più robusta, ad esempio integrazione OAuth2/social login (django-allauth in Django, Authlib o Flask-Dance in Flask). Impara a proteggere l'app da attacchi comuni: injection (l'ORM già aiuta prevenendo SQL injection), XSS (visto), CSRF (visto), e altri come clickjacking (Django invia header X-Frame-Options di default). - **Front-end e UX:** Un buon sviluppatore web comprende anche come migliorare l'esperienza utente: ciò può includere l'uso di JavaScript per feedback immediati, l'utilizzo di librerie CSS (Bootstrap, Tailwind) per layout responsivi, e eventualmente la separazione front-end/back-end con API REST se il progetto lo richiede. - **Deployment professionale:** Prova servizi come Heroku (anche se il piano gratuito classico è stato dismesso), DigitalOcean App Platform, AWS Elastic Beanstalk, Google App Engine, o container su AWS/GCP/Azure. Ogni piattaforma ha le sue particolarità, ma l'esperienza su PythonAnywhere/Render ti ha già introdotto concetti generici (wsgi, var env, static). - **Continuare a imparare:** Il panorama web evolve: ad esempio, i framework

async come FastAPI stanno guadagnando terreno per API ad alte prestazioni, oppure l'architettura a microservizi e l'uso di message broker (RabbitMQ via Celery per task asincroni in Django/Flask). Questi potrebbero essere temi futuri nel tuo percorso.

Infine, contribuisci o leggi codice di progetti open source in Flask/Django – è uno dei modi migliori per vedere pattern architetturali, trucchi e stile di codice in applicazioni reali. Buon coding e buona sperimentazione: la pratica continua è la chiave per padroneggiare lo sviluppo web Python!

Conclusione e Ringraziamenti

Complimenti per aver seguito questa lezione completa sullo sviluppo web essenziale con Python utilizzando Flask e Django! Abbiamo esplorato concetti teorici, esempi pratici e buone pratiche che ti forniranno una solida base per creare le tue applicazioni web. Ricorda che la differenza tra un programmatore alle prime armi e uno esperto spesso sta nella comprensione delle *best practice* e nell'esperienza nel risolvere problemi reali: continua quindi a sperimentare, sbagliare e correggere, perché è così che si consolida la competenza.

In conclusione, sia Flask che Django sono strumenti potenti. Scegli quello più adatto al progetto e alla tua mentalità di sviluppo: potresti persino usarli entrambi in contesti diversi. L'importante è aver acquisito la mentalità da sviluppatore web Python full-stack, capace di andare dall'idea al deploy. Non dimenticare la ricchezza della community: forum, documentazione e librerie sono i tuoi alleati quotidiani.

Grazie per l'attenzione e il tuo impegno. Auguro buon lavoro sui tuoi futuri progetti web in Python – che tu stia costruendo un piccolo sito personale o la prossima grande piattaforma, hai ora le conoscenze fondamentali per iniziare con il piede giusto. *Happy coding!*

```
# Grazie per aver seguito la lezione!  
print("Fine della lezione - in bocca al lupo con Flask e Django!")
```